a cyberwitchery talk

network automation for the modern age

veit heller
cyberwitchery lab

denog 18

section 01

01 hi.

i’m veit

- i build software, often network automation or security things.
- don’t let me near a router cli.
- i’ve been doing network automation for entirely too long.

i’m grateful for







today’s menu

- past and present (how we got here)
- an idea (the model in the middle)
- an example (alembic)
- a demo (deploy & drift)
- tradeoffs (no free lunch, yo)

section 02

02 recap

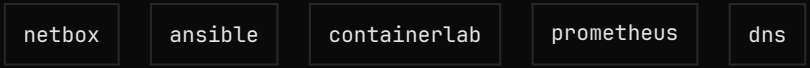
by hand

- ssh into the box, type the config.
- fat-finger something, find out later.
- forget what we did and why.
- learn how to resuce things really well.

write it down and script it

- document the setup in a **source of truth** (often >1).
- write small automations against it using salt, ansible, templates.
- this is where most of us still are today.

what that left us



- bespoke code in the middle.
- too many systems that don't talk to each other.
- our model has to fit the vendor world.

the vendor answer

buy another box that talks to their stack, and to the systems they consider important.

works as long as you stay inside their world.

section 03

03

a model of our own

the model we actually work from

> the source of truth's object model,
> plus custom fields and plugins,
> plus the spreadsheet on a samba share,
> plus concepts that only live in people's heads.

i think most of us end up with our own model anyway.

an example

```
dcim.device:
  key: { name: { type: slug } }
  fields:
    name:  { type: slug }
    kind:  { type: string }
    image: { type: string }
    cmd:   { type: string }
```

a lab node needs a kind, an image, and a command. netbox has none of them, so they end up in custom fields, config contexts, or a script.

the model is already there

every integration already decides what devices to look at, which fields matter, and what gets dropped.

the model exists somewhere in the aether.

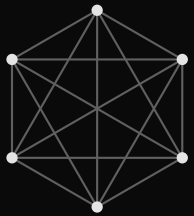
section 04

04 the model in the middle

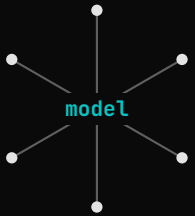
a different center

put your data model in the middle, and transform every system into and out of it.

one transform per system



point to point: n × m



model in the middle: n

a new system is one transform (or two), not one per system it talks to.

every operation is a transform

- **`deploy`**: netbox → containerlab, ansible, prometheus.
- **`drift`**: containerlab → netbox (just check).
- **`migration`**: netbox → nautobot. (you can even run both for a while, then cut over.)

it's all the same thing!

a compiler for your infra

many frontends, many backends.

the part that unifies them is the data and its types.

what you need is a system that is good at defining data models and transforming between them.

section 05

05

alembic

a case study

- our attempt at the idea, to see whether it holds up.
- **open core**: model, engine, and dcim / ipam adapters.
- commercial adapters for fabric and ops.

this is just one way to try it.

set the shape once

```
types:
  dcim.device:
    key:
      name:   { type: slug }
    fields:
      name:   { type: string }
      site:   { type: ref, target: dcim.site }
      role:   { type: ref, target: dcim.device_role }
      status: { type: string }
```

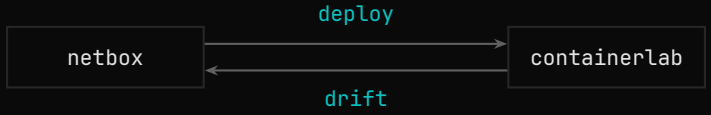
your types and fields and reference.

total control

```
$ alembic plan -f core-v1.yaml -o plan.json
plan: 5 to create, 0 to update, 0 to delete
plan written to plan.json
```

reading, mapping, planning, and finally applying.

demo time!



deploy a lab from netbox, change the lab manually.

→ live

deploy & drift.

section 06

06

trade-offs

you own the engineering

- all in on a vendor: excellent interoperability out of the box as long as you color inside the lines.
- own the model, make it the shape you want, and **more decisions that you own**.
- with great power comes great responsbility.

reinventing the universe

a bespoke model can mean
modelling everything from scratch.

expressiveness and recipes help, but teams can get stuck in design limbo.

it only pays off at a certain size

that size gets **smaller** as tools get better.

it’s never zero.

takeaways

- if you own the model, the transforms get easier.
- don't focus on the boxes, focus on **the edges between them**.
- think about the shape of your data, and don't take anything as gospel.

thanks.

questions?

veit heller veit@cyberwitchery.com

github.com/cyberwitchery/alembic

